

Salt-finger_convection_using_single-mode_equations

June 16, 2024

1 Salt-finger convection based on Single-mode equation using Dedalus 3

This notebook presents a Python code for simulating salt-finger convections in works by [Chang Liu 2022 JFM](#) by using [Dedalus framework](#) which performs the spectral method. In particular, we consider a fluid between two infinitely long parallel plates with a distance h . The temperature (T_*) and salinity S_* at these two plates are maintained at constant values with the top plate maintained at a higher temperature and salinity. The subscript $*$ denotes a dimensional variable. Variables are non-dimensionalized based on the thermal diffusivity κ_T

$$T = \frac{T_*}{\Delta T}, \quad S = \frac{S_*}{\Delta S}, \quad t = \frac{t_*}{h^2/\kappa_T}, \quad \vec{u} = \frac{\vec{u}_*}{\kappa_T/h}, \quad p = \frac{p_*}{\kappa_T^2 \rho_{r*}/h^2}. \quad (1)$$

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import dedalus.public as d3
```

```
2024-06-16 15:54:19,269 dedalus 0/1 WARNING :: Threading has not been disabled.
This may massively degrade Dedalus performance.
```

```
2024-06-16 15:54:19,272 dedalus 0/1 WARNING :: We strongly suggest setting the
"OMP_NUM_THREADS" environment variable to "1".
```

```
2024-06-16 15:54:21,672 numexpr.utils 0/1 INFO :: NumExpr defaulting to 4
threads.
```

```
/home/ducubuntu/miniforge-pypy3/lib/python3.9/site-
packages/numpy/core/getlimits.py:542: UserWarning: Signature
b'\x00\xd0\xcc\xcc\xcc\xcc\xcc\xcc\xfb\xbf\x00\x00\x00\x00\x00' for <class
'numpy.longdouble'> does not match any known type: falling back to type probe
function.
```

```
This warnings indicates broken support for the dtype!
```

```
machar = _get_machar(dtype)
```

In this work, I use a computational domain of $[L_x \times L_y \times L_z] := [2\pi/k_x \times 2\pi/k_y \times 1]$ with (k_x, k_y) representing the wavenumber pair in the horizontal directions. However, the present code only shows a 2D configuration ($k_y = 0$). Fourier and Shebyshev spectral methods are used in the horizontal and vertical directions with $N_x = 128$ and $N_z = 128$, respectively. Dealiasing scaling factor is set by $3/2$.

```
[2]: # Parameters
    kx, ky = 1, 0                # wavenumber
    Lx, Lz = 2.*np.pi/kx, 1.    # computational domain
    Nx, Nz = 128, 128           # number of points
    dealias = 3/2                # scaling factor
```

The system is governed by (equation 2.2 in the reference)

$$\nabla \cdot \mathbf{u} = 0, \quad (2)$$

$$\partial_t \mathbf{u} + \mathbf{u} \cdot \nabla \mathbf{u} = Pr \nabla^2 \mathbf{u} - \nabla p + Pr Ra_T (T - R_\rho^{-1} S) \mathbf{e}_z, \quad (3)$$

$$\partial_t T + \mathbf{u} \cdot \nabla T + w = \nabla^2 T, \quad (4)$$

$$\partial_t S + \mathbf{u} \cdot \nabla S + w = \tau \nabla^2 S. \quad (5)$$

where $\mathbf{u} = (u, v, w)$, p , and $\mathbf{e}_z$ are the fluid velocity, pressure, unit vector in the vertical. The governing parameters include the Prandtl number, the diffusivity ratio, the density ratio and the thermal Rayleigh number defined by

$$Ra := \frac{g\alpha\Delta T h^3}{\nu\kappa_T}, \quad Pr := \frac{\nu}{\kappa_T}, \quad \tau := \frac{\kappa_S}{\kappa_T}, \quad R_\rho := \frac{\alpha\Delta T}{\beta\Delta S}, \quad (6)$$

where ν is the viscosity and κ_S is the salinity diffusivity.

1.1 Single-mode equation for SFC

This section describes the governing equation by using the single-mode equation. Following the procedure in [Herring 1963](#), [Gough et al. 1975](#), [Gough & Toomre 1982](#), [Paparella & Spiegel 1999](#), and [Paparella et al. 2002](#), the single-mode ansatz is expressed by

$$S(x, y, z, t) = \bar{S}_0(z, t) + \hat{S}(z, t)e^{i(k_x x + k_y y)} + \text{c.c.}, \quad (7)$$

$$T(x, y, z, t) = \bar{T}_0(z, t) + \hat{T}(z, t)e^{i(k_x x + k_y y)} + \text{c.c.}, \quad (8)$$

$$\mathbf{u}(x, y, z, t) = \bar{U}_0(z, t)\mathbf{e}_x + \hat{\mathbf{u}}(z, t)e^{i(k_x x + k_y y)} + \text{c.c.}, \quad (9)$$

$$p(x, y, z, t) = \bar{P}_0(z, t) + \hat{p}(z, t)e^{i(k_x x + k_y y)} + \text{c.c.}, \quad (10)$$

with c.c. representing the complex conjugate. Velocity vector can be decomposed into

$$u(x, y, z, t) = \bar{U}_0(z, t) + \hat{u}(z, t)e^{i(k_x x + k_y y)} + \text{c.c.}, \quad (11)$$

$$v(x, y, z, t) = \hat{v}(z, t)e^{i(k_x x + k_y y)} + \text{c.c.}, \quad (12)$$

$$w(x, y, z, t) = \hat{w}(z, t)e^{i(k_x x + k_y y)} + \text{c.c.}, \quad (13)$$

Quantities are decomposed into a horizontal averaged component marked by $\bar{\cdot}$ and a single harmonic in the horizontal direction associated with the wavenumber pair (k_x, k_y) and characterised by the complex amplitude marked by $\hat{\cdot}$. For 2D configuration, velocity can be decomposed into a large-scale shear $\bar{U}_0(z, t)\mathbf{e}_x$ and a harmonic with the same wavenumber pair (k_x, k_y) . This allows generating a mean flow in the horizontal direction by assuming that the large-scale shear $\bar{\mathbf{u}}_0$ is generated in the

x-direction. **In three dimensions, the large-scale shear can be oriented in principle in any horizontal direction, a possibility that is left for future study.**

Now, we substitute above equations into the governing equation mentioned.

$$\begin{aligned} \partial_t \mathbf{u} + \mathbf{u} \cdot \nabla \mathbf{u} &= Pr \nabla^2 \mathbf{u} - \nabla p + Pr Ra_T (T - R_\rho^{-1} S) \mathbf{e}_z, \\ \Rightarrow \underbrace{\partial_t (\bar{U}_0 \mathbf{e}_x + \hat{\mathbf{u}} e^{i(k_x x + k_y y)})}_{A_1} + \underbrace{(\bar{U}_0 \mathbf{e}_x + \hat{\mathbf{u}} e^{i(k_x x + k_y y)}) \cdot \nabla (\bar{U}_0 \mathbf{e}_x + \hat{\mathbf{u}} e^{i(k_x x + k_y y)})}_{A_2} &= Pr \underbrace{\nabla^2 (\bar{U}_0 \mathbf{e}_x + \hat{\mathbf{u}} e^{i(k_x x + k_y y)})}_{A_4} \\ &\quad - \underbrace{\nabla (\bar{P}_0 + \hat{p} e^{i(k_x x + k_y y)})}_{A_5} + Pr Ra_T \underbrace{((\bar{T}_0 + \hat{T} e^{i(k_x x + k_y y)}) - R_\rho^{-1} (\bar{S}_0 + \hat{S} e^{i(k_x x + k_y y)})) \mathbf{e}_z}_{A_6}, \end{aligned}$$

Let us separately consider components

$$A_1 = \partial_t \bar{U}_0 \mathbf{e}_x + \partial_t \hat{\mathbf{u}} e^{i(k_x x + k_y y)},$$

$$\begin{aligned} A_2 &= \bar{U}_0 \mathbf{e}_x + \hat{\mathbf{u}} e^{i(k_x x + k_y y)} = (\bar{U}_0 + \hat{u} e^{i(k_x x + k_y y)}, \hat{v} e^{i(k_x x + k_y y)}, \hat{w} e^{i(k_x x + k_y y)}) \\ &= (\bar{U}_0 + \hat{u} e^{i(k_x x + k_y y)}, 0, \hat{w} e^{i(k_x x + k_y y)}), \end{aligned}$$

The salinity equation

$$\begin{aligned} \partial_t S + \mathbf{u} \cdot \nabla S + w &= \tau \nabla^2 S, \\ \Rightarrow \underbrace{\partial_t (\bar{S}_0 + \hat{S} e^{i(k_x x + k_y y)})}_{A_1} + \underbrace{(\bar{U}_0 \mathbf{e}_x + \hat{\mathbf{u}} e^{i(k_x x + k_y y)}) \cdot \nabla (\bar{S}_0 + \hat{S} e^{i(k_x x + k_y y)})}_{A_2} &+ \underbrace{\hat{w} e^{i(k_x x + k_y y)}}_{A_3} \\ &= \tau \underbrace{\nabla^2 (\bar{S}_0 + \hat{S} e^{i(k_x x + k_y y)})}_{A_4}, \end{aligned}$$

Let us separately consider components

$$A_1 = \partial_t \bar{S}_0 + \partial_t \hat{S} e^{i(k_x x + k_y y)},$$

$$\begin{aligned} A_2 &= \bar{U}_0 \mathbf{e}_x + \hat{\mathbf{u}} e^{i(k_x x + k_y y)} = (\bar{U}_0 + \hat{u} e^{i(k_x x + k_y y)}, \hat{v} e^{i(k_x x + k_y y)}, \hat{w} e^{i(k_x x + k_y y)}) \\ &= (\bar{U}_0 + \hat{u} e^{i(k_x x + k_y y)}, 0, \hat{w} e^{i(k_x x + k_y y)}), \end{aligned}$$

$$\begin{aligned} A_3 &= \nabla (\bar{S}_0 + \hat{S} e^{i(k_x x + k_y y)}) = (\partial_x (\bar{S}_0 + \hat{S} e^{i(k_x x + k_y y)}), \partial_y (\bar{S}_0 + \hat{S} e^{i(k_x x + k_y y)}), \partial_z (\bar{S}_0 + \hat{S} e^{i(k_x x + k_y y)})) \\ &= (\hat{S} \partial_x e^{i(k_x x + k_y y)}, \hat{S} \partial_y e^{i(k_x x + k_y y)}, \partial_z \bar{S}_0 + \partial_z \hat{S} e^{i(k_x x + k_y y)}) \\ &= (ik_x \hat{S} e^{i(k_x x + k_y y)}, ik_y \hat{S} e^{i(k_x x + k_y y)}, \partial_z \bar{S}_0 + \partial_z \hat{S} e^{i(k_x x + k_y y)}), \end{aligned}$$

$$\Rightarrow A_2 \cdot A_3 = (\bar{U}_0 + \hat{u} e^{i(k_x x + k_y y)}, 0, \hat{w} e^{i(k_x x + k_y y)}) \cdot (ik_x \hat{S} e^{i(k_x x + k_y y)}, ik_y \hat{S} e^{i(k_x x + k_y y)}, \partial_z \bar{S}_0 + \partial_z \hat{S} e^{i(k_x x + k_y y)})$$

$$\begin{aligned}
&= (\bar{U}_0 + \hat{u}e^{i(k_x x + k_y y)})(ik_x \hat{S}e^{i(k_x x + k_y y)}) + \hat{w}e^{i(k_x x + k_y y)}(\partial_z \bar{S}_0 + \partial_z \hat{S}e^{i(k_x x + k_y y)}) \\
&= ik_x \bar{U}_0 \hat{S}e^{i(k_x x + k_y y)} + ik_x \hat{u} \hat{S} + \partial_z \bar{S}_0 \hat{w}e^{i(k_x x + k_y y)} + \hat{u} \partial_z \hat{S},
\end{aligned}$$

$$\begin{aligned}
A_4 &= \nabla^2(\bar{S}_0 + \hat{S}e^{i(k_x x + k_y y)}) = (\partial_x^2 + \partial_y^2)(\hat{S}e^{i(k_x x + k_y y)}) + \partial_z^2(\bar{S}_0 + \hat{S}e^{i(k_x x + k_y y)}) \\
&= -(k_x^2 + k_y^2)\hat{S}e^{i(k_x x + k_y y)} + \partial_z^2 \bar{S}_0 + \partial_z^2 \hat{S}e^{i(k_x x + k_y y)},
\end{aligned}$$

So, the salinity equation becomes

$$\begin{aligned}
&\partial_t \bar{S}_0 + \partial_t \hat{S}e^{i(k_x x + k_y y)} + ik_x \bar{U}_0 \hat{S}e^{i(k_x x + k_y y)} + ik_x \hat{u} \hat{S} + \partial_z \bar{S}_0 \hat{w}e^{i(k_x x + k_y y)} + \hat{u} \partial_z \hat{S} + \hat{w}e^{i(k_x x + k_y y)} \\
&= -(k_x^2 + k_y^2)\tau \hat{S}e^{i(k_x x + k_y y)} + \partial_z^2 \tau \bar{S}_0 + \partial_z^2 \tau \hat{S}e^{i(k_x x + k_y y)}, \\
&\Rightarrow \partial_t \bar{S}_0 + \hat{u} \partial_z \hat{S} + ik_x \hat{u} \hat{S} + (\partial_t \hat{S} + ik_x \bar{U}_0 \hat{S} + \partial_z \bar{S}_0 \hat{w} + \hat{w})e^{i(k_x x + k_y y)} \\
&= \partial_z^2 \tau \bar{S}_0 + \tau(-(k_x^2 + k_y^2) + \partial_z^2)\hat{S}e^{i(k_x x + k_y y)},
\end{aligned}$$

Balance the horizontally averaged components, we have:

$$\boxed{\partial_t \bar{S}_0 + ik_x \hat{u} \hat{S} + \hat{u} \partial_z \hat{S} = \tau \partial_z^2 \bar{S}_0}, \quad (14)$$

Balance the harmonic components, we have:

$$\begin{aligned}
&\partial_t \hat{S} + ik_x \bar{U}_0 \hat{S} + \partial_z \bar{S}_0 \hat{w} + \hat{w} = \tau(-(k_x^2 + k_y^2) + \partial_z^2)\hat{S}, \\
&\Leftrightarrow \boxed{\partial_t \hat{S} + ik_x \bar{U}_0 \hat{S} + \partial_z \bar{S}_0 \hat{w} + \hat{w} = \tau \hat{\nabla}^2 \hat{S}},
\end{aligned}$$

with $\hat{\nabla}^2 = \partial_z^2 - k_x^2 - k_y^2$.

Similar transformation for temperature's governing equation, we obtained:

$$\begin{aligned}
&\partial_t \bar{T}_0 + \hat{u} \partial_z \hat{T} + ik_x \hat{u} \hat{T} + (\partial_t \hat{T} + ik_x \bar{U}_0 \hat{T} + \partial_z \bar{T}_0 \hat{w} + \hat{w})e^{i(k_x x + k_y y)} \\
&= \partial_z^2 \tau \bar{T}_0 + \tau(-(k_x^2 + k_y^2) + \partial_z^2)\hat{T}e^{i(k_x x + k_y y)}.
\end{aligned}$$

Then, we have

$$\boxed{\partial_t \bar{T}_0 + ik_x \hat{u} \hat{T} + \hat{u} \partial_z \hat{T} = \tau \partial_z^2 \bar{T}_0}, \quad (15)$$

and

$$\boxed{\partial_t \hat{T} + ik_x \bar{U}_0 \hat{T} + \partial_z \bar{T}_0 \hat{w} + \hat{w} = \tau \hat{\nabla}^2 \hat{T}}, \quad (16)$$

Finally, we have new governing equations obtained from original governing equations and single-mode equations. Terms of the vertical velocity w and vertical vorticity $\zeta := \partial_y u - \partial_x v$:

$$\begin{aligned}
\partial_t \hat{\nabla}^2 \hat{w} + ik_x \bar{U}_0 \hat{\nabla}^2 \hat{w} - ik_x \bar{U}_0'' \hat{w} &= \text{Pr} \hat{\nabla}^4 \hat{w} + \text{Pr} \hat{\nabla}_\perp^2 \text{Ra} (\hat{T} - R_\rho^{-1} \hat{S}), \\
\partial_t \hat{\zeta} + ik_x \bar{U}_0 \hat{\zeta} + ik_y \bar{U}_0' \hat{w} &= \text{Pr} \hat{\nabla}^2 \hat{\zeta}, \\
\partial_t \hat{T} + ik_x \bar{U}_0 \hat{T} + \hat{w} \partial_z \bar{T}_0 + \hat{w} &= \hat{\nabla}^2 \hat{T}, \\
\partial_t \hat{S} + ik_x \bar{U}_0 \hat{S} + \hat{w} \partial_z \bar{S}_0 + \hat{w} &= \tau \hat{\nabla}^2 \hat{S}, \\
\partial_t \bar{U}_0 + \partial_z (\hat{w}^* \bar{u} + \hat{w} \bar{u}^*) &= \text{Pr} \partial_z^2 \bar{U}_0, \\
\partial_t \bar{T}_0 + \partial_z (\hat{w}^* \bar{T} + \hat{w} \bar{T}^*) &= \partial_z^2 \bar{T}_0, \\
\partial_t \bar{S}_0 + \partial_z (\hat{w}^* \bar{S} + \hat{w} \bar{S}^*) &= \tau \partial_z^2 \bar{S}_0, \\
\hat{u} = \frac{ik_x \partial_z \hat{w}}{k_x^2 + k_y^2} - \frac{ik_y \hat{\zeta}}{k_x^2 + k_y^2}, \quad \hat{v} &= \frac{ik_y \partial_z \hat{w}}{k_x^2 + k_y^2} + \frac{ik_x \hat{\zeta}}{k_x^2 + k_y^2},
\end{aligned}$$

where the superscript $*$ denotes a complex conjugate and $\hat{\nabla}^2 := \partial_z^2 - k_x^2 - k_y^2$, $\hat{\nabla}_\perp^2 := -k_x^2 - k_y^2$, $\hat{\nabla}^4 := \partial_z^4 - 2(k_x^2 + k_y^2) \partial_z^2 + (k_x^2 + k_y^2)^2$, $\bar{U}_0' := \partial_z \bar{U}_0$ and $\bar{U}_0'' := \partial_z^2 \bar{U}_0$.

```
[3]: Ra = 1e5      # Rayleigh number
     Pr = 0.05    # Prandtl number
     tau = 0.01  # diffusivity ratio = ks/kt
     Rp = 40.0   # density ratio
```

To begin with, we must create basis for problem, including the mesh system based on the 2D Cartesian coordinate system and definition of field distributor with floating point in double precision.

```
[4]: # Bases
     # coords = d3.CartesianCoordinates('x', 'z')
     coords = d3.Coordinate('z')
     dist = d3.Distributor(coords, dtype=np.complex128)
```

There is a periodic boundary condition in horizontal direction, so we use Fourier for this direction (x axis). The no-periodic in vertical direction (z axis), the method of Chebyshev will be used there.

```
[5]: # define the coordinate system
     # xbasis = d3.ComplexFourier(coords['x'], size=Nx, bounds=(0, Lx),
     ↪ dealias=dealias)
     zbasis = d3.ChebyshevT(coords, size=Nz, bounds=(0, Lz), dealias=dealias)
```

Now, we will instantiate fields which will appears within the problem using the distributor, including pressure p , velocity $\vec{u}$, temperature T , and salinity S .

Following table shows representations of quantities in code.

Variables	Representations	Notes
u	u	Horizontal velocity component
$\bar{U}_0$	barU0	Horizontally averaged large-scale shear

Variables	Representations	Notes
$\hat{u}$	hatu	Amplitude of horizontal velocity's harmonic component
$\hat{w}$	hatw	Amplitude of vertical velocity's harmonic component
$\hat{\zeta}$	hatzeta	Amplitude of vertical vorticity's harmonic component
T	T	Temperature
$\bar{T}_0$	barT0	Horizontally averaged large-scale temperature component
$\hat{T}$	hatT	Amplitude of temperature's harmonic component
S	S	Sanility
$\bar{S}_0$	barS0	Horizontally averaged large-scale sanility component
$\hat{S}$	hatS	Amplitude of sanility's harmonic component

```
[6]: # define fields for single-mode equations
barU0 = dist.Field(name='barU0', bases=(zbasis))
hatu  = dist.Field(name='hatu', bases=(zbasis))
hatw  = dist.Field(name='hatw', bases=(zbasis))
hatzeta = dist.Field(name='hatzeta', bases=(zbasis))
barT0  = dist.Field(name='barT0', bases=(zbasis))
hatT   = dist.Field(name='hatT', bases=(zbasis))
barS0  = dist.Field(name='barS0', bases=(zbasis))
hatS   = dist.Field(name='hatS', bases=(zbasis))

# define output fields for analysis and visualization
# u = dist.Field(name='u', bases=(xbasis,zbasis))
# T = dist.Field(name='T', bases=(xbasis,zbasis))
# S = dist.Field(name='S', bases=(xbasis,zbasis))

tau_barU0 = dist.Field(name='tau_barU0')
tau_hatu  = dist.Field(name='tau_hatu')
tau_hatw  = dist.Field(name='tau_hatw')
tau_hatzeta = dist.Field(name='tau_hatzeta')
tau_barT0  = dist.Field(name='tau_barT0')
tau_hatT   = dist.Field(name='tau_hatT')
tau_barS0  = dist.Field(name='tau_barS0')
tau_hatS   = dist.Field(name='tau_hatS')
```

To simply when typing and determining the equations in Dedalus3, we can define replace operators for complex operators

```
[7]: # Substitutions
z = dist.local_grids(zbasis) # get coordinate arrays in horizontal and vertical
    ↪ directions
# print(z)
# ex, ez = coords.unit_vector_fields(dist) # get unit vectors in horizontal and
    ↪ vertical directions
i = 1j
conj = lambda A: np.conj(A)
```

```
dz = lambda A: d3.Differentiate(A, coords)
lift_basis = zbasis.derivative_basis(1)
lift = lambda A: d3.Lift(A, lift_basis, -1)
```

$$\begin{aligned}\hat{\nabla}^2 \hat{w} &:= \partial_z^2 \hat{w} - (k_x^2 + k_y^2) \hat{w}, \\ \hat{\nabla}_\perp^2 &:= -k_x^2 - k_y^2, \\ \bar{U}_0'' &:= \partial_z^2 \bar{U}_0, \\ \hat{\nabla}^4 &:= \partial_z^4 - 2(k_x^2 + k_y^2) \partial_z^2 + (k_x^2 + k_y^2)^2,\end{aligned}$$

```
[8]: kx2ky2 = kx*kx+ky*ky
hatnabla2 = lambda A: (dz(dz(A)) - kx2ky2*A)
hatnabla2perp = (-kx2ky2)
hatnabla4 = lambda A: (dz(dz(dz(dz(A)))) - 2*kx2ky2*dz(dz(A)) + kx2ky2*kx2ky2*A)
```

1.2 Define problem's equations

When we have all the fields for solving, we must build the IVP problem and introduce fields to the solver. At this step, we don't have full field for solving all three governing equations, so we just introduce corresponding codes to perform this.

```
[9]: # Problem
problem = d3.IVP([barU0, tau_barU0,
                  hatu, tau_hatu,
                  hatw, tau_hatw,
                  hatzeta, tau_hatzeta,
                  barS0, tau_barS0,
                  hatS, tau_hatS,
                  barT0, tau_barT0,
                  hatT, tau_hatT], namespace=locals())
```

$$\partial_t \hat{\nabla}^2 \hat{w} + i k_x \bar{U}_0 \hat{\nabla}^2 \hat{w} - i k_x \bar{U}_0'' \hat{w} = \text{Pr} \hat{\nabla}^4 \hat{w} + \text{Pr} \hat{\nabla}_\perp^2 \text{Ra} (\hat{T} - R_\rho^{-1} \hat{S}),$$

```
[10]: # equation 1
problem.add_equation("dt(hatnabla2(hatw)) - Pr*hatnabla4(hatw) -
    ↳ Pr*hatnabla2perp*Ra*(hatT-(1./Rp)*hatS) + lift(tau_hatw) = -
    ↳ i*kx*barU0*hatnabla2(hatw) + i*kx*dz(dz(barU0))*hatw")
```

```
[10]: {'LHS': Add(Add(Add(Convert(dt(Add(Diff(Diff(<Field 140322475087616>))),
    Convert(Mul(-1, <Field 140322475087616>))))), Mul(-1, Mul(0.05,
    Add(Add(Diff(Diff(Diff(Diff(<Field 140322475087616>))))), Convert(Mul(-1, Mul(2,
    Diff(Diff(<Field 140322475087616>))))))), Convert(<Field 140322475087616>))))),
    Convert(Mul(-1, Mul(-5000.0, Add(<Field 140322475087376>, Mul(-1, Mul(0.025,
    <Field 140322475089344>))))))), Convert(Mul(<Field 140322475118656>, <Field
    140322475119760>))))),
```

```

'RHS': Add(Mul(Mul(-1j, <Field 140322475086080>), Add(Diff(Diff(<Field
140322475087616>))), Convert(Mul(-1, <Field 140322475087616>))))), Mul(Mul(1j,
Diff(Diff(<Field 140322475086080>))), <Field 140322475087616>)),
'condition': 'True',
'tensorsig': (),
'dtype': dtype('complex128'),
'M': Convert(Add(Diff(Diff(<Field 140322475087616>))), Convert(Mul(-1, <Field
140322475087616>))))),
'L': Add(Add(Mul(-1, Mul(0.05, Add(Add(Diff(Diff(Diff(Diff(<Field
140322475087616>))))), Convert(Mul(-1, Mul(2, Diff(Diff(<Field
140322475087616>))))))), Convert(<Field 140322475087616>))), Convert(Mul(-1,
Mul(-5000.0, Add(<Field 140322475087376>, Mul(-1, Mul(0.025, <Field
140322475089344>))))))), Convert(Mul(<Field 140322475118656>, <Field
140322475119760>))),
'F': Convert(Add(Mul(Mul(-1j, <Field 140322475086080>), Add(Diff(Diff(<Field
140322475087616>))), Convert(Mul(-1, <Field 140322475087616>))))), Mul(Mul(1j,
Diff(Diff(<Field 140322475086080>))), <Field 140322475087616>))),
'domain': <dedalus.core.domain.Domain at 0x7f9f5f4811f0>,
'matrix_dependence': array([ True]),
'matrix_coupling': array([ True])}

```

$$\begin{aligned}
\partial_t \hat{\zeta} + ik_x \bar{U}_0 \hat{\zeta} + ik_y \bar{U}'_0 \hat{w} &= \text{Pr} \hat{\nabla}^2 \hat{\zeta}, \\
\partial_t \hat{T} + ik_x \bar{U}_0 \hat{T} + \hat{w} \partial_z \bar{T}_0 + \hat{w} &= \hat{\nabla}^2 \hat{T}, \\
\partial_t \hat{S} + ik_x \bar{U}_0 \hat{S} + \hat{w} \partial_z \bar{S}_0 + \hat{w} &= \tau \hat{\nabla}^2 \hat{S},
\end{aligned}$$

with $\bar{U}'_0 := \partial_z \bar{U}_0$

```

[11]: # equation 2
problem.add_equation("dt(hatzeta) - Pr*hatnabla2(hatzeta) + lift(tau_hatzeta) =_
    ↪ i*kx*barU0*hatzeta - i*kx*dz(barU0)*hatw")
# equation 3
problem.add_equation("dt(hatT) + hatw - hatnabla2(hatT) + lift(tau_hatT) = -_
    ↪ i*kx*barU0*hatT - hatw*dz(barT0)")
# # equation 4
problem.add_equation("dt(hatS) + hatw - tau*hatnabla2(hatS) + lift(tau_hatS) =_
    ↪ i*kx*barU0*hatS - hatw*dz(barS0)")

```

```

[11]: {'LHS': Add(Add(Convert(Add(dt(<Field 140322475089344>), <Field
140322475087616>))), Mul(-1, Mul(0.01, Add(Diff(Diff(<Field 140322475089344>))),
Convert(Mul(-1, <Field 140322475089344>))))), Convert(Mul(<Field
140322475118656>, <Field 140322475119328>))),
'RHS': Add(Mul(Mul(-1j, <Field 140322475086080>), <Field 140322475089344>),
Mul(-1, Mul(<Field 140322475087616>, Diff(<Field 140322475086224>))))),
'condition': 'True',
'tensorsig': (),
'dtype': dtype('complex128'),

```



```

'M': Convert(<Field 140322475089344>),
'L': Add(Add(Convert(<Field 140322475087616>), Mul(-1, Mul(0.01,
Add(Diff(Diff(<Field 140322475089344>))), Convert(Mul(-1, <Field
140322475089344>))))), Convert(Mul(<Field 140322475118656>, <Field
140322475119328>))),
'F': Convert(Add(Mul(Mul(-1j, <Field 140322475086080>), <Field
140322475089344>), Mul(-1, Mul(<Field 140322475087616>, Diff(<Field
140322475086224>))))),
'domain': <dedalus.core.domain.Domain at 0x7f9f5f481e80>,
'matrix_dependence': array([ True]),
'matrix_coupling': array([ True])}

```

$$\begin{aligned}
\partial_t \bar{U}_0 + \partial_z (\hat{w}^* \hat{u} + \hat{w} \hat{u}^*) &= \text{Pr} \partial_z^2 \bar{U}_0, \\
\partial_t \bar{T}_0 + \partial_z (\hat{w}^* \hat{T} + \hat{w} \hat{T}^*) &= \partial_z^2 \bar{T}_0, \\
\partial_t \bar{S}_0 + \partial_z (\hat{w}^* \hat{S} + \hat{w} \hat{S}^*) &= \tau \partial_z^2 \bar{S}_0,
\end{aligned}$$

```

[12]: # equation 5
problem.add_equation("dt(barU0) - Pr*dz(dz(barU0)) + lift(tau_barU0) = -_
    ↳dz(conj(hatw)*hatu+hatw*conj(hatu))")
# equation 6
problem.add_equation("dt(barT0) - dz(dz(barT0)) + lift(tau_barT0)= -_
    ↳dz(conj(hatw)*hatT+hatw*conj(hatT))")
# equation 7
problem.add_equation("dt(barS0) - tau*dz(dz(barS0)) + lift(tau_barS0)= -_
    ↳dz(conj(hatw)*hatS+hatw*conj(hatS))")

```

```

[12]: {'LHS': Add(Add(Convert(dt(<Field 140322475086224>)), Mul(-1, Mul(0.01,
Diff(Diff(<Field 140322475086224>))))), Convert(Mul(<Field 140322475118656>,
<Field 140322475119424>))),
'RHS': Mul(-1, Diff(Add(Mul(conjugate(<Field 140322475087616>), <Field
140322475089344>), Mul(<Field 140322475087616>, conjugate(<Field
140322475089344>))))),
'condition': 'True',
'tensorsig': (),
'dtype': dtype('complex128'),
'M': Convert(<Field 140322475086224>),
'L': Add(Mul(-1, Mul(0.01, Diff(Diff(<Field 140322475086224>))))),
Convert(Mul(<Field 140322475118656>, <Field 140322475119424>))),
'F': Convert(Mul(-1, Diff(Add(Mul(conjugate(<Field 140322475087616>), <Field
140322475089344>), Mul(<Field 140322475087616>, conjugate(<Field
140322475089344>))))),
'domain': <dedalus.core.domain.Domain at 0x7f9f5f481e80>,
'matrix_dependence': array([ True]),
'matrix_coupling': array([ True])}

```

$$\hat{u} = \frac{ik_x \partial_z \hat{w}}{k_x^2 + k_y^2} - \frac{ik_y \hat{\zeta}}{k_x^2 + k_y^2},$$

$$\hat{v} = \frac{ik_y \partial_z \hat{w}}{k_x^2 + k_y^2} + \frac{ik_x \hat{\zeta}}{k_x^2 + k_y^2},$$

```
[13]: # equation 5
problem.add_equation("hatu = i*kx*dz(hatw)/(kx*kx+ky*ky) - i*ky*hatzeta/
↪(kx*kx+ky*ky)")
```

```
[13]: {'LHS': <Field 140323698493760>,
'RHS': Mul(1j, Diff(<Field 140322475087616>)),
'condition': 'True',
'tensorsig': (),
'dtype': dtype('complex128'),
'M': 0,
'L': Convert(<Field 140323698493760>),
'F': Mul(1j, Diff(<Field 140322475087616>)),
'domain': <dedalus.core.domain.Domain at 0x7f9f5f4818e0>,
'matrix_dependence': array([ True]),
'matrix_coupling': array([ True])}
```

1.3 Set up the boundary conditions

Researching vertically confined salt-finger convection helps people to understand the interior between two well-mixed layers. The temperature and salinity are imposed as constant variables rolling boundary conditions in this simulation. The corresponding boundary conditions for the salinity and temperature are:

$$\begin{aligned} \hat{S}(z=0, t) = \hat{S}(z=1, t) = \hat{T}(z=0, t) = \hat{T}(z=1, t) \\ = \bar{S}_0(z=0, t) = \bar{S}_0(z=1, t) = \bar{T}_0(z=0, t) = \bar{T}_0(z=1, t) = 0, \end{aligned}$$

```
[14]: # Boundary conditions
problem.add_equation("barS0(z=0) = 0")
problem.add_equation("barS0(z=Lz) = 0")
problem.add_equation("hatS(z=0) = 0")
problem.add_equation("hatS(z=Lz) = 0")

problem.add_equation("barT0(z=0) = 0")
problem.add_equation("barT0(z=Lz) = 0")
problem.add_equation("hatT(z=0) = 0")
problem.add_equation("hatT(z=Lz) = 0")
```

```
[14]: {'LHS': interp(<Field 140322475087376>, z=1.0),
'RHS': 0,
'condition': 'True',
```

```

'tensorsig': (),
'dtype': numpy.complex128,
'M': 0,
'L': interp(<Field 140322475087376>, z=1.0),
'F': <Field 140322474700512>,
'domain': <dedalus.core.domain.Domain at 0x7f9f5f4894c0>,
'matrix_dependence': array([ True]),
'matrix_coupling': array([ True])}

```

To approach laboratory experiments more, the no-slip boundary condition for velocity is adopted at top and bottom planes. In some cases, we can use the stress-free velocity boundary conditions instead to understand oceanographic scenarios. The no-slip boundary condition for velocity in present situation is defined by

$$\begin{aligned}\hat{w}(z=0, t) = \hat{w}(z=1, t) = \partial_z \hat{w}(z=0, t) = \partial_z \hat{w}(z=1, t) \\ = \hat{\zeta}(z=0, t) = \hat{\zeta}(z=1, t) = \bar{U}_0(z=0, t) = \bar{U}_0(z=1, t) = 0.\end{aligned}$$

```

[15]: problem.add_equation("barU0(z=0) = 0")
      problem.add_equation("barU0(z=Lz) = 0")
      problem.add_equation("hatw(z=0) = 0")
      problem.add_equation("hatw(z=Lz) = 0")
      problem.add_equation("hatszeta(z=0) = 0")
      problem.add_equation("hatszeta(z=Lz) = 0")
      problem.add_equation("dz(hatw)(z=0) = 0") # ?
      problem.add_equation("dz(hatw)(z=Lz) = 0") # ?

```

```

[15]: {'LHS': interp(Diff(<Field 140322475087616>), z=1.0),
      'RHS': 0,
      'condition': 'True',
      'tensorsig': (),
      'dtype': numpy.complex128,
      'M': 0,
      'L': interp(Diff(<Field 140322475087616>), z=1.0),
      'F': <Field 140322474799936>,
      'domain': <dedalus.core.domain.Domain at 0x7f9f5f4894c0>,
      'matrix_dependence': array([ True]),
      'matrix_coupling': array([ True])}

```

1.4 Build Solver

```

[16]: stop_sim_time = 300 # Stopping criteria
      timestepper = d3.RK443 # 3rd-order 4-stage DIRK+ERK scheme [Ascher 1997 sec 2.
      ↪8] https://doi-org.ezproxy.lib.uconn.edu/10.1016/S0168-9274(97)00056-1
      # timestepper = d3.RK222
      # Solver
      solver = problem.build_solver(timestepper)
      solver.stop_sim_time = stop_sim_time

```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[16], line 5
      2 timestepper = d3.RK443 # 3rd-order 4-stage DIRK+ERK scheme [Ascher 1997,
    ↪sec 2.8] https://doi-org.ezproxy.lib.uconn.edu/10.1016/S0168-9274(97)00056-1
      3 # timestepper = d3.RK222
      4 # Solver
----> 5 solver = problem.build_solver(timestepper)
      6 solver.stop_sim_time = stop_sim_time

File ~/miniforge-pypy3/lib/python3.9/site-packages/dedalus/core/problems.py:98,
    ↪in ProblemBase.build_solver(self, *args, **kw)
      96 def build_solver(self, *args, **kw):
      97     """Build corresponding solver class."""
----> 98     return self.solver_class(self, *args, **kw)

File ~/miniforge-pypy3/lib/python3.9/site-packages/dedalus/core/solvers.py:523,
    ↪in InitialValueSolver.__init__(self, problem, timestepper,
    ↪enforce_real_cadence, warmup_iterations, **kw)
      521 self.init_time = self.world_time
      522 # Build LHS matrices
--> 523 self.build_matrices(self.subproblems, ['M', 'L'])
      524 # Compute total modes
      525 local_modes = sum([prod(sp.shape) for sp in self.subproblems])

File ~/miniforge-pypy3/lib/python3.9/site-packages/dedalus/core/solvers.py:122,
    ↪in SolverBase.build_matrices(self, subproblems, matrices)
      120 if matrices is None:
      121     matrices = self.matrices
--> 122 subsystems.build_subproblem_matrices(self, subproblems, matrices)

File ~/miniforge-pypy3/lib/python3.9/site-packages/dedalus/core/subsystems.py:
    ↪81, in build_subproblem_matrices(solver, subproblems, matrices)
      79 # Build matrices
      80 for subproblem in log_progress(subproblems, logger, 'info',
    ↪desc='Building subproblem matrices', iter=np.inf, frac=0.1, dt=10):
----> 81     subproblem.build_matrices(matrices)

File ~/miniforge-pypy3/lib/python3.9/site-packages/dedalus/core/subsystems.py:
    ↪545, in Subproblem.build_matrices(self, names)
      543 # Check squareness of restricted system
      544 if valid_eqn.nnz != valid_var.nnz:
--> 545     raise ValueError("Non-square system: group={}, I={}, J={}".
    ↪format(self.group, valid_eqn.nnz, valid_var.nnz))
      547 # Permutations
      548 left_perm = left_permutation(self, eqns, bc_top=solver.bc_top,
    ↪interleave_components=solver.interleave_components).tocsr()

```

```
ValueError: Non-square system: group=(None,), I=1040, J=1032
```

1.4.1 Set up initial conditions

```
[ ]: barU0.fill_random('g', seed=42, distribution='normal', scale=1e-3) # Random noise
      hatu.fill_random('g', seed=42, distribution='normal', scale=1e-3) # Random noise
      hatw.fill_random('g', seed=42, distribution='normal', scale=1e-3) # Random noise
      hatzeta.fill_random('g', seed=42, distribution='normal', scale=1e-3) # Random noise
      barS0.fill_random('g', seed=42, distribution='normal', scale=1e-3) # Random noise
      hatS.fill_random('g', seed=42, distribution='normal', scale=1e-3) # Random noise
      barT0.fill_random('g', seed=42, distribution='normal', scale=1e-3) # Random noise
      hatT.fill_random('g', seed=42, distribution='normal', scale=1e-3) # Random noise
```

```
[ ]: # Analysis
      import shutil, os
      sim_name = 'SMESFC_DNS'
      # if os.path.exists(sim_name):
      #     shutil.rmtree(sim_name) # remove the output directory and its previous contents

      # dataset = solver.evaluator.add_file_handler(sim_name, sim_dt=1.0, max_writes=1000)
      # dataset.add_task(barU0, name='barU0')
      # dataset.add_task(hatu, name='hatu')
      # dataset.add_task(hatw, name='hatw')
      # dataset.add_task(barS0, name='barS0')
      # dataset.add_task(hatS, name='hatS')
      # dataset.add_task(barT0, name='barT0')
      # dataset.add_task(hatT, name='hatT')
```

```
[ ]: max_timestep = 0.125
      # CFL
      CFL = d3.CFL(solver, initial_dt=max_timestep, cadence=10, safety=0.5, threshold=0.05,
                  max_change=1.5, min_change=0.5, max_dt=max_timestep)
      # CFL.add_velocity(barU0+hatu*np.exp(i*(kx*x)))
```

```
[ ]: # Main loop
      print('Starting main loop')
      while solver.proceed:
          timestep = CFL.compute_timestep()
```

```

print(timestep)
solver.step(timestep)
if (solver.iteration-1) % 1000 == 0:
    print('Completed iteration {}, time={:.3f}'.format(solver.iteration,
↪solver.sim_time))
    sa_plot = plt.pcolormesh(np.copy(barS0['g']).transpose())
    plt.colorbar(sa_plot)
    plt.title("t = {:.3f}".format(solver.sim_time))
    plt.savefig("salinity{:.3f}.png".format(solver.sim_time), dpi=200)
    plt.close()

```